

Adaptive Extensions to Autoformer for Long-Term Time Series Forecasting

Fariba Ghorbani

M.Sc. Student in Computer Science

University of Tehran

faribaqorbani@ut.ac.ir

Abstract

Long-term time series forecasting requires efficient modeling of long-range and periodic dependencies. Autoformer addresses this problem through progressive series decomposition and FFT-based Auto-Correlation, but its decomposition uses a fixed moving-average scale, its training-time speedup can share Top- k delays across a batch, and selected delays are weighted directly by correlation strength. This work investigates three modifications: **Adaptive Multi-Scale Decomposition**, which learns a fusion of moving averages with window sizes 13, 25, and 49. **Sample-Adaptive Lag Selection**, which selects Top- k delays independently for each sample; and **Adaptive Period Routing**, which uses a lightweight neural router to adjust candidate-period weights.

Experiment 2A evaluates sample-adaptive selection alone, while 2A+2B applies selection and routing together. On multivariate ETTm2 with an input length of 96 and prediction horizons of 96, 192, 336, and 720, 2A+2B achieves the best overall performance, reducing average MSE from approximately 0.324 to 0.313 and MAE from 0.365 to 0.355. adaptive Multi-Scale Decomposition improves the 96-step forecast but degrades performance at longer horizons, suggesting that sample-adaptive period modeling is the more consistent modification.

1 Introduction

Time series forecasting aims to predict future observations from historical measurements arranged in temporal order. It is important in many practical applications, including energy management, transportation, economics, weather prediction, and industrial monitoring. In these applications, extending the prediction horizon is valuable because decisions often need to be made considerably before the future events of interest occur.

Long-term forecasting, however, is considerably more difficult than short-term forecasting. A time series usually contains several interacting behaviors, such as slowly changing trends, periodic patterns, local fluctuations, and irregular noise. When the prediction horizon becomes longer, these components become harder to distinguish, and small modeling errors can have a larger effect on the final prediction.

Deep neural networks, especially Transformer-based models, have provided effective approaches for modeling long-range dependencies. The Transformer [2] uses self-attention to directly connect distant positions in a sequence. This ability is useful for time series, where relevant observations may be separated by many time steps. However, standard self-attention requires $O(L^2)$ computation and memory for a sequence of length L . This becomes expensive when long input or output sequences are considered.

Several Transformer variants have therefore focused on reducing the cost of self-attention. LogTrans [5], Reformer [4], and Informer [3], for example, introduce sparse or approximate attention mechanisms. Although these methods improve efficiency, the underlying dependency modeling remains largely point-wise: the model identifies relationships between individual positions and aggregates information from selected positions.

Autoformer [1] approaches the forecasting problem from a different direction. Instead of only making attention sparse, Autoformer introduces two time-series-specific ideas into the Transformer architecture. First, it embeds **series decomposition** directly inside the encoder and decoder so that trend and seasonal information are progressively separated during forecasting. Second, it replaces self-attention with an **Auto-Correlation**

mechanism, which discovers periodic relationships and aggregates entire shifted sub-series rather than isolated time points.

These properties make Autoformer a suitable starting point for studying long-term forecasting. Nevertheless, examination of its implementation also reveals several design choices that may limit adaptation to individual input windows.

The first concerns decomposition. The original decomposition block extracts the trend component using a single fixed moving-average window. A fixed window assumes that one smoothing scale is suitable for different variables, different samples, and different intermediate representations inside the model. For a multivariate dataset such as ETTm2, this assumption may be restrictive. Some variables may contain relatively fast changes, while others may follow slower operational patterns.

A second limitation appears in the efficient training implementation of Auto-Correlation. Auto-Correlation initially computes correlation information for each sample. However, during training, correlations are averaged over the batch before the Top- k delay indices are selected. Consequently, all samples in that batch can be forced to use the same candidate periods. If different windows contain different dominant periodicities, averaging their correlation profiles can hide sample-specific peaks.

A third issue concerns the weighting of the selected periods. Once Autoformer finds the Top- k delays, their correlation values are used directly to determine their aggregation weights. This is a sensible starting point, since a strongly correlated historical pattern is likely to be relevant. However, **correlation strength is not necessarily identical to forecasting usefulness**. A previously strong period may become less useful when recent behavior changes.

Based on these observations, this study investigates three modifications to Autoformer:

1. **Adaptive Multi-Scale Decomposition**, which replaces one fixed moving-average scale with an adaptive combination of several

scales.

2. **Sample-Adaptive Lag Selection (2A)**, which performs Top- k delay selection independently for each sample instead of after batch averaging.
3. **Adaptive Period Routing (2B)**, which introduces a small learnable router that adjusts the importance of the candidate periods.

The experimental design for the last two modifications is important. Experiment 2A evaluates Sample-Adaptive Lag Selection by itself. The second configuration, denoted **2A+2B**, retains the sample-specific lag selection of 2A and additionally applies Adaptive Period Routing. Therefore, the reported 2A+2B results should not be interpreted as an isolated evaluation of 2B.

Experiments are conducted on the ETTm2 dataset using a look-back length of 96 and prediction lengths of 96, 192, 336, and 720. The results show that the period-adaptation modifications provide the most consistent gains. Sample-Adaptive Lag Selection performs particularly well for the 96-step horizon, while the combined 2A+2B model obtains the best results for horizons 192, 336, and 720.

The main contributions of this work are therefore as follows:

- We study a multi-scale extension of Autoformer's progressive decomposition mechanism using adaptive fusion of short-, medium-, and long-range moving averages.
 - We modify the training-time Auto-Correlation procedure so that candidate periods can be selected independently for each input sample.
 - We introduce a lightweight period router that learns a correction to the original correlation-based period importance.
 - We experimentally compare the reproduced Autoformer baseline and the proposed modifications on ETTm2 and analyze both successful and unsuccessful architectural changes rather than assuming that increased model flexibility always improves forecasting.
-

3 Autoformer Architecture

3.1 Forecasting Problem

Following the notation of Autoformer, the forecasting task can be described as predicting a future sequence of length O from a historical sequence of length I . For the experiments in this work,

$$I = 96,$$

while

$$O \in \{96, 192, 336, 720\}.$$

For ETTm2, observations are recorded every 15 minutes. Therefore, 96 observations correspond to approximately one day of historical data. The prediction horizons consequently range from one day to substantially longer future intervals.

3.2 Progressive Series Decomposition

A major difference between Autoformer and ordinary Transformer models is that decomposition is not used only as preprocessing.

For an intermediate representation

$$X \in \mathbb{R}^{L \times d}$$

the original series decomposition block calculates the trend-cyclical component using a moving average:

$$X_t = \text{AvgPool}(\text{Padding}(X))$$

The seasonal component is then obtained by subtraction:

$$X_s = X - X_t.$$

Therefore,

$$X = X_s + X_t.$$

Here, X_t represents the smoother trend-cyclical part, whereas X_s contains the remaining seasonal or relatively higher-frequency variation.

The importance architectural idea is that this operation appears repeatedly inside Autoformer. The encoder progressively removes trend information from intermediate hidden representations, while the decoder separately maintains and accumulates trend information. This design is described by the original paper as **progressive decomposition**.

Encoder

The Autoformer encoder mainly model the seasonal representation. In simplified form, an encoder layer first applies Auto-Correlation with a residual connection,

$$H^{(l,1)} = \text{AutoCorrelation}(X^{(l-1)}) + X^{(l-1)},$$

followed by decomposition:

$$S^{(l,1)}, T^{(l,1)} = \text{SeriesDecomp}(H^{(l,1)}).$$

The trend output is remove from the encoder path. the remaining seasonal representation passes through the feed forward network and another decomposition operation. as a result, deeper encoder layers operate on increasingly refined seasonal information.

Decoder

The decoder contains both seasonal and trend paths.

Its seasonal path includes:

1. self Auto-Correlation,
2. encoder-decoder Auto-Correlation,
3. a feed-forward block,

with decomposition performed after these transformations
unlike the encoder, the decodr does not discard the extracted trend components. Instead, they are progressivly projected and added to the decoder's trend representation.

The final output can therefore be viewed approximately as

$$\hat{Y} = \hat{Y}_{seasonal} + \hat{Y}_{trend}.$$

This structure is important for the proposed Adaptive Multi-Scale Decomposition because changing the decomposition operator effects several locations inside the model, while the general encoder-decoder logic remains unchanged.

3.3 Auto-Correlation Mechanism

Instead of asking which individual time positions should attend to each other, Auto-Correlation asks a more time-series-oriented question:

If one sequence is shifted by a delay τ , how similar is it to another sequence?

For a simple discrete sequence, correlation at lag τ can be written conceptually as

$$R(\tau) = \sum_t X_t X_{t-\tau}.$$

In Autoformer, query and key representations are used, giving

$$R_{Q,K}(\tau)$$

Large values indicate that the two representations contain similar temporal processes when one is shifted by τ .

Autoformer then identifies the strongest k delays:

$$\tau_1, \dots, \tau_k = \arg \operatorname{TopK}_{\tau} R_{Q,K}(\tau)$$

where

$$k = \lfloor c \log L \rfloor$$

and c is a hyperparameter.

The selected correlation values are normalized using softmax:

$$\alpha_i = \frac{\exp(R_{Q,K}(\tau_i))}{\sum_{j=1}^k \exp(R_{Q,K}(\tau_j))}$$

Finally, the Value sequence is shifted according to each selected delay and aggregated:

$$\text{AutoCorrelation}(Q, K, V) = \sum_{i=1}^k \alpha_i \text{Roll}(V, \tau_i)$$

This mechanism aggregates entire shifted sub-series instead of selecting isolated positions. the original Autoformer paper describes this as **series-wise** rather than point-wise dependency modeling.

3.4 Efficient Correlation with FFT

Calculating every lag independently in the time domain would be expensive. Autoformer avoids this by computing the correlation in the frequency domain.

For Query and Key sequences,

$$Q_f = \mathcal{F}(Q)$$

$$K_f = \mathcal{F}(K)$$

where $\mathcal{F}$ denotes the Fourier transform.

Their frequency-domain product is computed using the complex conjugate of the Key spectrum:

$$S_{Q,K} = Q_f \odot K_f^*$$

The correlation sequence over all lag is then obtained using the inverse Fourier transform:

$$R_{Q,K} = \mathcal{F}^{-1} (Q_f \odot K_f^*)$$

this follows the Wiener-Khinchin relationship [7].

FFT therefore allows Autoformer to obtain the correlation values of all candidate lags together, leading to approximately

$$O(L \log L)$$

complexity rather than explicitly evaluating all L^2 pairwise temporal relationship

The propose Experiments 2A and 2B retains this FFT computation. Their changes occur **after the correlation profile has been computed**, during the selection and weighting of delays.

4 Proposed Modifications

4.1 Adaptive Multi-Scale Decomposition

4.1.1 Motivation

The original Autoformer uses one moving-average scale for decomposition. in the implementation considered in this work, the standard scale is 25:

$$T = \text{MA}_{25}(X)$$

$$S = X - T$$

this is simple and computationally efficcent, but it assumes that one smoothing scale is sutable for different signals and hidden representations.

This assumption can be restrictive in a multivariate dataset. A relatively small moving-average window follows local changes more closely, whereas a larger window produces a smoother estimate of long-term behavior.

For example:

- a short window may preserve relatively fast changes in the trend;
- a medium window may provide a balance between local and global behavior;

- a long window may represent slowly changing operational conditions. The modification therefore attempts to let the model determine how much each scale should contribute.

4.1.2 Multi-Scale Trend Candidates

Three moving-average operations are applied:

$$T^{(1)} = \text{MA}_{13}(X)$$

$$T^{(2)} = \text{MA}_{25}(X)$$

$$T^{(3)} = \text{MA}_{49}(X)$$

These provide short, medium and longer-scale trend candidates. Instead of selecting one scale manually, a softmax gate produces weights

$$\alpha_1, \alpha_2, \alpha_3$$

such that

$$\alpha_1 + \alpha_2 + \alpha_3 = 1$$

The adaptive trend becomes

$$T = \alpha_1 T^{(1)} + \alpha_2 T^{(2)} + \alpha_3 T^{(3)}$$

or equivalently

$$T = \alpha_1 \text{MA}_{13}(X) + \alpha_2 \text{MA}_{25}(X) + \alpha_3 \text{MA}_{49}(X)$$

The seasonal component is calculated in the same way as in the original Autoformer:

$$S = X - T$$

The workflow is therefore

$$X \rightarrow \{\text{MA}_{13}, \text{MA}_{25}, \text{MA}_{49}\} \rightarrow \text{Softmax Gate} \rightarrow \text{Weighted Trend} \rightarrow X - T$$

4.1.3 Integration with Autoformer

An important goal was to preserve the original logic of Autoformer. The adaptive module still return

$$(S, T) = \text{SeriesDecomp}(X)$$

Therefore, the surrounding encoder and decoder structure does not need to be redesigned.

Progressive decomposition remains active, Auto-Correlation continues to operate on seasonal representations, and the decoder continues accumulating the trend components.

The modification can therefore be summarized as

$$\text{MA}_{25}(X)$$

being replaced by

$$\alpha_1 \text{MA}_{13}(X) + \alpha_2 \text{MA}_{25}(X) + \alpha_3 \text{MA}_{49}(X)$$

The hypothesis is that this adaptive decomposition could avoid under-smoothing or over-smoothing and provide cleaner seasonal information to later Auto-Correlation layers.

4.2 Sample-Adaptive Lag Selection: Experiment 2A

4.2.1 Limitation of the Original Training Procedure

Auto-Correlation initially produces correlation values separately for each input sample.

For a multi-head representation, the correlation tensor can be written schematically as

$$C \in \mathbb{R}^{B \times H \times E \times L}$$

where

- B is batch size,
- H is the number of head,
- E represents features within a head,
- L is the number of candidate delays.

The head and feature dimensions can be averaged to obtain

$$R \in \mathbb{R}^{B \times L}$$

Thus each sample initially has its own correlation profile.

However, in the original training-time speedup procedure, Autoformer performs another average across the batch:

$$\bar{R}(\tau) = \frac{1}{B} \sum_{b=1}^B R_b(\tau)$$

This change the representation from

$$[B, L]$$

to

$$[L]$$

Top- k is then performed on the batch-averaged profile:

$$D = \text{TopK}(\bar{R})$$

Consequently, all samples use the same selected delay indices during that training step.

For example, one sample may prefer lags $\{2, 4\}$, another may prefer $\{1, 3\}$, but the averaged profile may chose $\{2, 3\}$. In that case, both samples lose one individually useful delay.

4.2.2 Proposed Per-Sample Top- k

Experiment 2A removes the batch averaging operation before Top- k .

The original path can be summarized as

$$[B, H, E, L] \rightarrow [B, L] \rightarrow [L] \rightarrow [K]$$

Experiment 2A instead uses

$$[B, H, E, L] \rightarrow [B, L] \rightarrow [B, K]$$

Formally, for sample b ,

$$D_b = \text{TopK}(R_b(\tau))$$

Therefore,

$$D = \{D_1, D_2, \dots, D_B\}$$

One sample may select

$$D_1 = \{24, 48\}$$

while another may select

$$D_2 = \{30, 60\}$$

The Value sequence is then shifted according to the delays selected for that sample.

The rest of Auto-correlation remains unchanged. In particular, experiment 2A does **not** introduce a new learnable module. Selected delays are still weighted using their correlation strengths.

For sample b ,

$$\alpha_{b,i} = \text{Softmax}_i(R_b(\tau_{b,i})),$$

and

$$Y_b = \sum_{i=1}^k \alpha_{b,i} \text{Roll}(V_b, \tau_{b,i}).$$

Thus, Experiment 2A changes the **route through Auto-Correlation**, not the basic mechanism itself.

This is also a useful controlled ablation because no extra parameters are added. Any performance difference comes primarily from retaining sample-specific period candidates.

4.3 Adaptive Period Routing: Experiment 2B

4.3.1 Motivation

Experiments 2A addresses which delays should be selected. Experiment 2B addresses a different question:

Once useful candidate delays have been selected, should correlation strength alone determine how much each delay contributes?

In the original mechanism,

$$\alpha_i = \text{Softmax}(R(\tau_i))$$

Therefore, if

$$R(\tau_1) > R(\tau_2)$$

then τ_1 generally receives a larger aggregation weight.

This assumes that

$$\text{correlation strength} \approx \text{forecasting usefulness}$$

This assumption is reasonable but not always guaranteed.

For example, historical data may show strong periodicity at lag 2, but if recent behavior has changed, lag 4 may provide information that is more relevant to the future even though its overall correlation value is lower.

4.3.2 Learnable Period Router

Adaptive Period Routing introduces a small multilayer perceptron after the candidate delays have been selected.

Instead of using only

$$R(\tau_i)$$

the final score for candidate i is

$$s_i = R(\tau_i) + g_\theta(\phi_i)$$

where

- $R(\tau_i)$ is the original Auto-Correlation strength,
- g_θ is the learnable period router,
- ϕ_i describes the current sample and candidate delay.

The final period weights become

$$\alpha_i = \text{Softmax}(s_i)$$

The original correlation score therefore remains present. The router learns only an adjustment rather than completely replacing auto-correlation.

4.3.3 Router Input

For each candidate lag the router receives 10 values.

Autocorrelation-profile summary

The full sample-specific correlation profile is compressed into eight values:

$$p = [p_1, p_2, \dots, p_8]$$

These values provide a compact summary of the overall periodic structure of the current sample.

Candidate correlation strength

The router also receives

$$R(\tau_i)$$

which tells it how strong the candidate is according to the original Auto-correlation mechanism.

Normalized lag position

finally, the candidate lag is normalized according to the sequence length:

$$\ell_i = \frac{\tau_i}{L - 1}$$

The complete feature vector is

$$\phi_i = [p_1, \dots, p_8, R(\tau_i), \ell_i]$$

Thus,

$$\phi_i \in \mathbb{R}^{10}$$

The implemented router has the structure

$$10 \rightarrow 16 \rightarrow 1$$

the same router is applied to each candidate period.

4.3.4 Residual Initialization

The router is designed as a correction to Experiment 2A.

At initialization,

$$g_\theta(\phi_i) \approx 0$$

therefore

$$s_i \approx R(\tau_i)$$

and

$$\alpha_i \approx \text{Softmax}(R(\tau_i))$$

The model consequently starts approximately from the behavior of 2A and learns adjustments during training rather than beginning with unrelated random period weights.

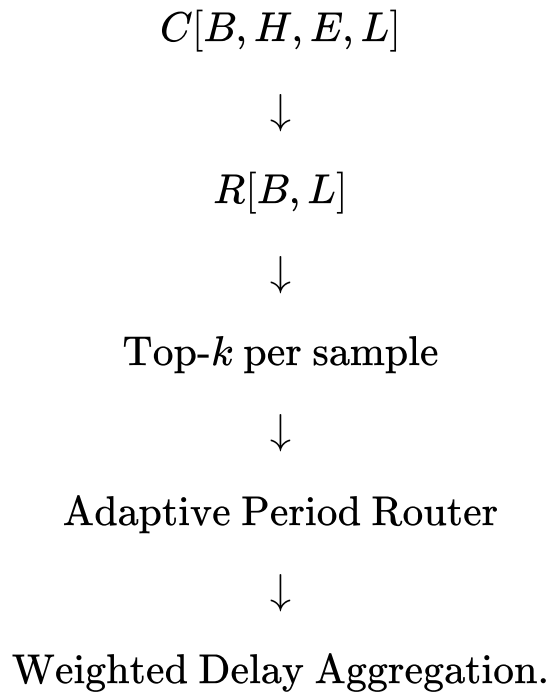
4.4 Combined 2A+2B Configuration

An important point in interpreting the experiments is that 2B is not reported as an isolated configuration.

Three configurations are compared:

Configuration	Lag selection	Lag weighting
Baseline	Original Autoformer training procedure	Auto-Correlation strength
2A	Per-sample Top- k	Auto-Correlation strength
2A+2B	Per-sample Top- k	Adaptive Period Router

Therefore, the complete combined process is



Experiment 2A determine which periods are candidates.

Experiment 2B determine how strongly the selected candidates should contribute.

5 Experiments

5.1 ETTm2 Dataset

The experiments are conducted on the **ETTm2** dataset from the Electricity Transformer Temperature benchmark.

The dataset contains measurements collected from an electricity transformer system at 15 minute intervals over approximately two years. It contains seven variables, including oil temperature and six power-load measurements.

Because the sampling interval is 15 minutes,

$$4 \times 24 = 96$$

measurements correspond to one day.

This makes a sequence length of 96 a natural look-back interval while allowing progressively longer prediction horizons to test long-term behavior.

5.2 Forecasting Setting

The historical input length is fixed at 96.

Four prediction horizons are evaluated:

$$96, 192, 336, 720$$

The experiments use the same general forecasting setup when comparing architectural variants so that the results can primarily be attributed to the proposed modifications rather than unrelated changes in the tasks.

The evaluated configurations are:

- reproduced baseline Autoformer;
 - Autoformer with Adaptive Multi-Scale Decomposition;
 - Autoformer with Sample-Adaptive Lag Selection (2A);
 - Autoformer with Sample-Adaptive Lag Selection and Adaptive Period Routing (2A+2B).
-

5.3 Evaluation Metrics

Two standard forecasting metrics are used: Mean Squared Error and Mean Absolute Error.

For N predictions,

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

and

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

lower values indicate better forecasting performance.

MSE gives larger penalties to large prediction errors, whereas MAE provides a more direct measurement of average absolute deviation.

Reporting both makes it easier to distinguish general improvement from improvement caused only by a small number of unusually large errors.

6 Results and Discussion

The experimental results are summarized below

Horizon	Baseline	Adaptive Multi-Scale	2A	2A+2B
96	0.247 / 0.324	0.239 / 0.316	0.221 / 0.307	0.232 / 0.313
192	0.282 / 0.341	0.283 / 0.345	0.280 / 0.340	0.272 / 0.330
336	0.331 / 0.367	0.404 / 0.406	0.331 / 0.366	0.323 / 0.360
720	0.435 / 0.429	0.476 / 0.450	0.425 / 0.419	0.423 / 0.418

Each entry is reported as **MSE / MAE**.

6.1 Adaptive Multi-Scale Decomposition

at predictions length 96, Adaptive Multi-Scale Decomposition improves both metrics:

$$\text{MSE} : 0.247 \rightarrow 0.239,$$

$$\text{MAE} : 0.324 \rightarrow 0.316.$$

This corresponds to approximately a 3.24% reduction in MSE and a 2.47% reduction in MAE.

This result supports part of the original motivation. at relatively short horizon, allowing the model to combine multiple smoothing scales may help it represent recent local behavior more accurately than a single moving average.

The situation changes for longer horizons.

At horizon 192, performance is nearly unchanged but slightly worse:

$$0.282/0.341 \rightarrow 0.283/0.345$$

At horizon 336, the degradation is considerable:

$$0.331/0.367 \rightarrow 0.404/0.406$$

At horizon 720

$$0.435/0.429 \rightarrow 0.476/0.450$$

The average values across the four horizons are approximately

$$\text{MSE}_{multi} = 0.351$$

$$\text{MAE}_{multi} = 0.379$$

compared with

$$\text{MSE}_{base} = 0.324$$

$$\text{MAE}_{base} = 0.365$$

Therefore, the multi-scale modification does not provide a general improvement.

This result is useful rather than simply negative. It suggests that Autoformer's fixed decomposition may provide a form of useful regularization. Decomposition affects the representation that later enters Auto-Correlation. If the decomposition boundary changes too freely between different samples or representations, some periodic information that should remain in the seasonal component may instead be absorbed into the trend, or vice versa.

The experiment therefore shows that **more adaptive decomposition is not automatically better decomposition**.

A natural next direction would be to constrain the fusion weights, analyze which scales are selected in different layers, or make the additional scales residual corrections around the original 25-step decomposition.

6.2 Sample-Adaptive Lag Selection

Experiment 2A gives much more consistent results.

At horizon 96,

$$\text{MSE} : 0.247 \rightarrow 0.221$$

which is approximately a 10.53% reduction.

MAE changes from

$$0.324 \rightarrow 0.307$$

or approximately 5.25% lower. This is the best 96-step result among all tested configurations. The improvements at horizon 192 is smaller:

$$0.282/0.341 \rightarrow 0.280/0.340$$

At 336, MSE is unchanged to the reported precision

$$0.331 \rightarrow 0.331$$

while MAE slightly improves:

$$0.367 \rightarrow 0.366$$

at the longest horizon, performance again improves more clearly:

$$0.435/0.429 \rightarrow 0.425/0.419$$

averaged over the four horizons:

$$\text{MSE}_{2A} = 0.3143$$

$$\text{MAE}_{2A} = 0.3580$$

Relative to the baseline averages, this corresponds to approximately 2.93% lower MSE and 1.98% lower MAE.

An important aspect of this result is that Experiment 2A does not require an additional learned module. The improvement comes from retaining information that was already available in the AutoCorrelation tensor but was removed by batch averaging before Top- k .

The results therefore provide evidence for the hypothesis that different input windows can benefit from different period candidates.

6.3 Combined Sample-Adaptive Lag Selection and Adaptive Period Routing

The combined 2A+2B configuration obtains the best overall performance. At horizon 96, it improves significantly over the baseline:

$$0.247/0.324 \rightarrow 0.232/0.313$$

however 2A alone remains better at this horizon. at horizon 192, the combined model gives 0.272/0.330 compared with 0.282/0.341 for the baseline.

This corresponds to reductions of approximately 3.55% in MSE and 3.23% in MAE. at horizon 336

$$0.331/0.367 \rightarrow 0.323/0.360$$

corresponding to approximately 2.42% lower MSE and 1.91% lower MAE.

At horizon 720,

$$0.435/0.429 \rightarrow 0.423/0.418$$

This gives approximately 2.76% lower MSE and 2.56% lower MAE.
The average performance is

$$\text{MSE}_{2A+2B} = 0.3125$$

$$\text{MAE}_{2A+2B} = 0.3553$$

Compared with the baseline averages of 0.3238 and 0.3653, respectively, the combined configuration reduces average MSE by approximately 3.47% and average MAE by approximately 2.74%. Thus, 2A+2B provides the lowest average error among the evaluated architectures.

6.4 Incremental Effect of Adaptive Period Routing

Because 2B is evaluated together with 2A the most useful comparison for understanding the router is

$$2A \quad \text{vs.} \quad 2A+2B$$

Horizon	2A MSE	2A+2B MSE	2A MAE	2A+2B MAE
96	0.221	0.232	0.307	0.313
192	0.280	0.272	0.340	0.330
336	0.331	0.323	0.366	0.360
720	0.425	0.423	0.419	0.418

At horizon 96, adding the router makes performance worse than 2A.

At horizons 192, 336, and 720, however, the router improves both MSE and MAE.

This pattern is interesting. For a shorter forecast, raw Auto-Correlation may already provide a sufficiently good indication of which selected

period should receive the largest weight. The router introduces additional flexibility that is not useful in this particular case.

For longer forecasts, the situation appears different. The strongest historical correlation may not always be the best guide for a more distant future. The router can adjust the relative importance of candidate periods using the broader correlation structure of the sample.

This interpretation is consistent with the motivation for Experiment 2B, although it should not be treated as proof of the mechanism. Further analysis of learned router outputs would be needed to determine exactly why particular periods are up-weighted or down-weighted.

6.5 Comparison of the Three Proposed Ideas

The experiments reveal an important difference between modifying decomposition and modifying period routing.

Adaptive Multi-Scale decomposition makes a relatively early and broad change to the architecture. Once the trend estimate changes the seasonal residual changes as well:

$$S = X - T$$

That modified representation then propagates through later Auto-Correlation and decoder operations. A poor decomposition decision can therefore affect several downstream computations.

Experiments 2A and 2B make more localized changes.

Experiment 2A asks:

Which periods should this sample use?

Experiment 2B asks:

How much should this sample trust each selected period?

The FFT computation, correlation representation, rolling mechanism, progressive decomposition, and overall encoder-decoder architecture remain essentially unchanged.

The experimental results suggest that this more focused adaptation is more reliable for ETTm2.

In particular:

- **2A is best for horizon 96.**
- **2A+2B is best for horizons 192, 336, and 720.**
- **Adaptive Multi-Scale Decomposition improves horizon 96 but degrades the longer horizons.**

This is a stronger research result than simply reporting that all architectural changes improve the model. It indicates which assumptions in Autoformer appear worth relaxing and which ones may require more careful modification.

7 Conclusion

This work studied three architectural modifications to Autoformer for long-term multivariate time series forecasting.

Autoformer provides a strong starting point because it incorporates time-series-specific structure directly into a Transformer-style encoder-decoder architecture. Progressive decomposition separates trend-cyclical and seasonal representations throughout the network, while Auto-Correlation replaces point-wise self-attention with period-based dependency discovery and sub-series aggregation. FFT allows correlations over all candidate delays to be computed efficiently in approximately $O(L \log L)$ time [1].

The first proposed modification, **Adaptive Multi-Scale Decomposition**, replaces the fixed moving-average scale with a learned fusion of moving averages with window sizes 13, 25, and 49. The modification improves the 96-step ETTm2 forecast but performs worse than the baseline at longer horizons. This suggests that adaptive decomposition is a reasonable idea, but the decomposition stage may require stronger constraints because changes at this stage affect all downstream seasonal and trend representations.

The second modification, **Sample-Adaptive Lag Selection**, addresses the batch-level averaging used before Top- k selection in the original training-time Auto-Correlation procedure. By retaining the batch dimension until after Top- k , each sample can use its own dominant delays. This relatively small, parameter-free modification improves average MSE and MAE and produces the best result for the 96-step horizon.

The third modification, **Adaptive Period Routing**, questions the assumption that Auto-Correlation strength alone should determine the importance of a selected delay. A small $10 \rightarrow 16 \rightarrow 1$ MLP uses a pooled summary of the sample's correlation profile, the candidate correlation value, and its normalized lag position to learn a residual adjustment to the original score.

In the experiments, Adaptive Period Routing is combined with Sample-Adaptive Lag Selection. The resulting **2A+2B** model achieves the best results at prediction horizons 192, 336, and 720. Across all four horizons, average MSE decreases from approximately 0.324 for the reproduced baseline to 0.313, while average MAE decreases from approximately 0.365 to 0.355. These correspond to reductions of approximately 3.47% and 2.74%, respectively.

Taken together, the results suggest that **sample-specific adaptation inside Auto-Correlation is a promising direction for extending Autoformer**. Instead of replacing Autoformer's central period-based inductive bias, the proposed modifications preserve it while making two decisions more flexible: which periods should be selected and how much each selected period should contribute.

Future work should evaluate these mechanisms on additional datasets, repeat experiments with multiple random seeds, study the learned lag distributions and router outputs, and investigate more constrained forms of multi-scale decomposition. Such analysis would help determine whether the improvements observed on ETTm2 represent a general advantage of sample-adaptive period modeling in long-term forecasting.

References

- [1] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting," *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 2017.
- [3] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting," *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.
- [4] N. Kitaev, Ł. Kaiser, and A. Levskaya, "Reformer: The Efficient Transformer," *International Conference on Learning Representations (ICLR)*, 2020.
- [5] S. Li, X. Jin, Y. Xuan, X. Zhou, W. Chen, Y.-X. Wang, and X. Yan, "Enhancing the Locality and Breaking the Memory Bottleneck of Transformer on Time Series Forecasting," *Advances in Neural Information Processing Systems*, 2019.
- [6] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2018.
- [7] N. Wiener, "Generalized Harmonic Analysis," *Acta Mathematica*, 1930.
- [8] G. E. P. Box and G. M. Jenkins, *Time Series Analysis, Forecasting and Control*, 1970.
- [9] B. N. Oreshkin, D. Carpow, N. Chapados, and Y. Bengio, "N-BEATS: Neural Basis Expansion Analysis for Interpretable Time Series Forecasting," *International Conference on Learning Representations*, 2019.